

BIBLE psutil & pynput – RÉFÉRENCE EXHAUSTIVE DES FONCTIONS

LÉGENDE : [ps] = psutil | [pn] = pynput | [proc] = Process | [kb] = keyboard
[ms] = mouse | [lst] = listener | [ctrl] = controller

IMPORTS ET PRÉSENTATION

DESCRIPTION : psutil (Process and System UTILities) fournit des informations sur les processus, le CPU, la mémoire, les disques et le réseau. pynput permet de contrôler et surveiller le clavier et la souris.
POURQUOI : Monitoring système, automatisation, tests, bots, outils DevOps.
CONTEXTE : Scripts de monitoring, automatisation de bureau, ergonomie.

IMPORTS :

```
import psutil                                Module principal
from pynput import keyboard, mouse           Modules clavier et souris
from pynput.keyboard import Key, KeyCode, Listener as KListener, Controller as KController
from pynput.mouse import Button, Listener as MListener, Controller as MController
```

COMPATIBILITÉ :

```
psutil    → Windows, Linux, macOS, FreeBSD, OpenBSD, NetBSD, Sun Solaris, AIX
pynput    → Windows, Linux (X11 ou uinput), macOS
# Linux pynput : nécessite DISPLAY ou accès /dev/input (root pour uinput)
# macOS pynput : nécessite les autorisations d'accessibilité (Préférences Système)
```

PSUTIL – SYSTÈME ET PROCESSUS

[psutil] CPU

DESCRIPTION : Informations sur le processeur : charge, fréquence, cœurs, stats.
POURQUOI : Monitoring de charge, détection de goulots, profiling système.
CONTEXTE : Dashboard de monitoring, alertes de performance, DevOps.

UTILISATION CPU :

```
psutil.cpu_percent()                % global (non bloquant, relatif au dernier appel)
psutil.cpu_percent(interval=1)      % global sur 1 seconde (bloquant)
psutil.cpu_percent(interval=0.5, percpu=True) % par cœur logique → [12.5, 8.0, ...]
psutil.cpu_percent(interval=None)    Depuis le dernier appel (non bloquant)
# Premier appel retourne toujours 0.0 (pas de référence précédente)
# Appeler deux fois avec interval=None : 1er=0.0, 2ème=vraie valeur
```

TEMPS CPU :

```
psutil.cpu_times()                  Temps CPU global (namedtuple)
t = psutil.cpu_times()
t.user                              Temps en mode utilisateur (secondes)
t.system                            Temps en mode noyau
t.idle                              Temps d'inactivité
t.nice                              Processus à priorité réduite (Unix)
t.iowait                             Attente I/O (Linux)
t.irq                               Interruptions matérielles (Linux/Windows)
t.softirq                           Interruptions logicielles (Linux)
t.steal                             Temps volé par l'hyperviseur (Linux)
t.guest                             Machine virtuelle invitée (Linux)
psutil.cpu_times(percpu=True)        → [namedtuple par cœur]
psutil.cpu_times_percent(interval=1) Même chose en pourcentages
psutil.cpu_times_percent(interval=1, percpu=True)
```

STATISTIQUES CPU :

```
psutil.cpu_stats()                  Statistiques CPU (namedtuple)
s = psutil.cpu_stats()
s.ctx_switches                      Nombre de changements de contexte
s.interrupts                        Nombre d'interruptions
s.soft_interrupts                   Interruptions logicielles (Linux)
s.syscalls                          Appels système (Windows/macOS)
psutil.cpu_freq()                   Fréquence CPU courante (namedtuple)
f = psutil.cpu_freq()
f.current                            Fréquence courante (MHz)
f.min                               Fréquence minimale
f.max                               Fréquence maximale
psutil.cpu_freq(percpu=True)        → [namedtuple par cœur]
```

NOMBRE DE CŒURS :

<code>psutil.cpu_count()</code>	Cœurs logiques (avec hyperthreading)
<code>psutil.cpu_count(logical=True)</code>	Idem (défaut)
<code>psutil.cpu_count(logical=False)</code>	Cœurs physiques uniquement

Exemple : 4 cœurs physiques + HT = `cpu_count()`=8, `cpu_count(False)`=4
Retourne None si impossible à déterminer

CHARGE SYSTÈME (Unix) :

<code>psutil.getloadavg()</code>	→ (1min, 5min, 15min) charge moyenne
----------------------------------	--------------------------------------

`load1, load5, load15 = psutil.getloadavg()`
Disponible sous Windows depuis psutil 5.6.2 (simulé)

EXEMPLES :

```
# Monitoring continu du CPU
import time
while True:
    pcts = psutil.cpu_percent(interval=1, percpu=True)
    print(" | ".join(f"CPU{i}: {p:5.1f}%" for i, p in enumerate(pcts)))
# Alerte si CPU > 90%
if psutil.cpu_percent(interval=2) > 90:
    print("⚠ CPU surchargé !")
```

[psutil] Mémoire

DESCRIPTION : Informations sur la mémoire RAM et le swap.
POURQUOI : Détection de fuites mémoire, monitoring, alertes.
CONTEXTE : Monitoring de serveurs, profiling d'applications.

MÉMOIRE VIRTUELLE (RAM) :

<code>psutil.virtual_memory()</code>	Statistiques RAM complètes (namedtuple)
<code>vm = psutil.virtual_memory()</code>	
<code>vm.total</code>	RAM totale (octets)
<code>vm.available</code>	RAM disponible (octets)
<code>vm.used</code>	RAM utilisée
<code>vm.free</code>	RAM libre (non utilisée et non mise en cache)
<code>vm.percent</code>	% d'utilisation
<code>vm.active</code>	Mémoire active (Linux/macOS)
<code>vm.inactive</code>	Mémoire inactive (Linux/macOS)
<code>vm.buffers</code>	Buffers noyau (Linux)
<code>vm.cached</code>	Mise en cache (Linux/macOS)
<code>vm.shared</code>	Mémoire partagée (Linux)
<code>vm.wired</code>	Mémoire câblée (macOS)
# Conversion en MB/GB	
<code>vm.total / (1024**2)</code>	En MB
<code>vm.total / (1024**3)</code>	En GB

SWAP :

<code>psutil.swap_memory()</code>	Statistiques swap (namedtuple)
<code>sw = psutil.swap_memory()</code>	
<code>sw.total</code>	Swap total (octets)
<code>sw.used</code>	Swap utilisé
<code>sw.free</code>	Swap libre
<code>sw.percent</code>	% d'utilisation du swap
<code>sw.sin</code>	Octets swappés depuis le disque (Linux)
<code>sw.sout</code>	Octets swappés vers le disque (Linux)

EXEMPLES :

```
vm = psutil.virtual_memory()
print(f"RAM: {vm.used/1e9:.1f} GB / {vm.total/1e9:.1f} GB ({vm.percent}%)")
if vm.percent > 85:
    print("⚠ Mémoire critique !")
```

[psutil] Disques

DESCRIPTION : Partitions, utilisation des disques, I/O.
POURQUOI : Surveiller l'espace disque, détecter les goulots d'I/O.
CONTEXTE : Monitoring de stockage, alertes, analyse de performance.

PARTITIONS :

<code>psutil.disk_partitions()</code>	Liste des partitions montées
<code>psutil.disk_partitions(all=False)</code>	Seulement les vraies partitions (défaut)
<code>psutil.disk_partitions(all=True)</code>	Toutes (y compris pseudo-FS)

```

for part in psutil.disk_partitions():
    part.device          Ex: "/dev/sda1" ou "C:\\\\"
    part.mountpoint      Ex: "/" ou "C:\\\\"
    part.fstype           Ex: "ext4", "ntfs", "apfs"
    part.opts             Options de montage
    part.maxfile          Longueur max de nom de fichier
    part.maxpath          Longueur max de chemin

UTILISATION DISQUE :
    psutil.disk_usage(path)          Utilisation d'une partition
    du = psutil.disk_usage("/")
    du.total                         Espace total (octets)
    du.used                         Espace utilisé
    du.free                         Espace libre
    du.percent                      % d'utilisation
    psutil.disk_usage("C:\\")       Windows
    psutil.disk_usage("/home")      Partition /home

I/O DISQUE :
    psutil.disk_io_counters()        Compteurs I/O globaux (namedtuple)
    io = psutil.disk_io_counters()
    io.read_count                   Nombre de lectures
    io.write_count                  Nombre d'écritures
    io.read_bytes                   Octets lus
    io.write_bytes                  Octets écrits
    io.read_time                    Temps de lecture (ms)
    io.write_time                   Temps d'écriture (ms)
    io.busy_time                    Temps d'activité (Linux, Windows)
    psutil.disk_io_counters(perdisk=True) → dict {nom_disque: namedtuple}
    psutil.disk_io_counters(perdisk=True)["sda"].read_bytes
    psutil.disk_io_counters(nowrap=True) Évite les overflows de compteurs

EXEMPLES :
    for part in psutil.disk_partitions():
        try:
            du = psutil.disk_usage(part.mountpoint)
            print(f"{part.mountpoint}: {du.percent}% "
                  f"({du.free/1e9:.1f} GB libres)")
        except PermissionError:
            pass

```

[psutil] Réseau

DESCRIPTION : Interfaces réseau, connexions, compteurs I/O.
POURQUOI : Monitoring réseau, audit de sécurité, débogage de connectivité.
CONTEXTE : Surveillance des connexions, monitoring de bande passante.

```

COMpteurs I/O RÉSEAU :
    psutil.net_io_counters()        Compteurs globaux (namedtuple)
    net = psutil.net_io_counters()
    net.bytes_sent                  Octets envoyés
    net.bytes_recv                  Octets reçus
    net.packets_sent                Paquets envoyés
    net.packets_recv                Paquets reçus
    net.errin                       Erreurs en entrée
    net.errout                      Erreurs en sortie
    net.dropin                      Paquets abandonnés en entrée
    net.dropout                     Paquets abandonnés en sortie
    psutil.net_io_counters(pernic=True) → dict {interface: namedtuple}
    psutil.net_io_counters(pernic=True)["eth0"].bytes_recv
    psutil.net_io_counters(nowrap=True)

INTERFACES RÉSEAU :
    psutil.net_if_addrs()           Adresses par interface → dict
    addrs = psutil.net_if_addrs()
    for iface, addr_list in addrs.items():
        for addr in addr_list:
            addr.family              socket.AF_INET (IPv4), AF_INET6, AF_PACKET
            addr.address              Adresse IP ou MAC
            addr.netmask              Masque de sous-réseau
            addr.broadcast            Adresse de broadcast
            addr.ptp                  Point-to-point (si applicable)
    psutil.net_if_stats()           Statuts des interfaces → dict
    stats = psutil.net_if_stats()

```

```

for iface, s in stats.items():
    s.isup          Interface active ?
    s.duplex        NIC_DUPLEX_FULL/HALF/UNKNOWN
    s.speed         Vitesse (Mbps)
    s.mtu           MTU

CONNEXIONS RÉSEAU :
psutil.net_connections()      Toutes les connexions (nécessite root Linux)
psutil.net_connections(kind="tcp") Connexions TCP seulement
psutil.net_connections(kind="tcp4") TCP IPv4
psutil.net_connections(kind="tcp6") TCP IPv6
psutil.net_connections(kind="udp")  UDP
psutil.net_connections(kind="unix") Sockets Unix
psutil.net_connections(kind="inet") TCP + UDP IPv4 + IPv6
psutil.net_connections(kind="all")  Tout
for conn in psutil.net_connections("tcp"):
    conn.fd          File descriptor
    conn.family      AF_INET, AF_INET6...
    conn.type        SOCK_STREAM, SOCK_DGRAM
    conn.laddr       Adresse locale (ip, port)
    conn.raddr       Adresse distante (ip, port)
    conn.status      "ESTABLISHED", "LISTEN", "TIME_WAIT"...
    conn.pid         PID du processus

EXEMPLES :
# Débit réseau sur 1 seconde
n1 = psutil.net_io_counters()
time.sleep(1)
n2 = psutil.net_io_counters()
print(f"↑ {(n2.bytes_sent-n1.bytes_sent)/1e3:.1f} KB/s "
      f"↓ {(n2.bytes_recv-n1.bytes_recv)/1e3:.1f} KB/s")

```

[psutil] Processus – liste et recherche

DESCRIPTION : Lister, rechercher et inspecter les processus du système.
POURQUOI : Gestion de processus, monitoring d'applications, automation.
CONTEXTE : Scripts d'administration, surveillance d'applications, CI/CD.

LISTE DES PROCESSUS :
 psutil.pids() Liste de tous les PIDs → [int]
 psutil.process_iter() Itérateur sur tous les processus
 psutil.process_iter(["pid", "name", "status"]) Avec attributs pré-chargés (efficace)
 for proc in psutil.process_iter(["pid", "name", "cpu_percent"]):
 print(proc.info) {"pid":123, "name":"python", ...}
 psutil.process_iter(["pid", "name", "memory_percent", "username"])
 # Toujours utiliser process_iter() avec une liste d'attrs pour les performances

PROCESSUS SPÉCIFIQUE :
 p = psutil.Process(pid) Depuis un PID
 p = psutil.Process() Processus courant (pid=os.getpid())
 psutil.pid_exists(pid) True si le PID existe
 # Éviter d'utiliser pid_exists() puis Process() (race condition)
 # Utiliser directement Process(pid) dans un try/except NoSuchProcess

RECHERCHE DE PROCESSUS :
 # Trouver par nom
 def find_process(name):
 for proc in psutil.process_iter(["pid", "name"]):
 if proc.info["name"] == name:
 return proc
 # Trouver par nom de commande
 [p for p in psutil.process_iter(["pid", "cmdline"])
 if "python" in (p.info["cmdline"] or [])]
 # Trouver par utilisateur
 [p.info for p in psutil.process_iter(["pid", "name", "username"])
 if p.info["username"] == "root"]

EXCEPTIONS :
 psutil.NoSuchProcess(pid) PID n'existe plus
 psutil.AccessDenied(pid) Permission refusée
 psutil.ZombieProcess(pid) Processus zombie (Unix)
 psutil.TimeoutExpired(seconds) Timeout dépassé

DESCRIPTION : Informations détaillées sur un processus individuel.
 POURQUOI : Profiling, débogage, monitoring granulaire.
 CONTEXTE : Analyse de performance, débogage de fuites, monitoring.

INFORMATIONS DE BASE :

p = psutil.Process(pid)	
p.pid	PID
p.ppid()	PID du parent
p.name()	Nom du processus (ex: "python3")
p.exe()	Chemin de l'exécutable
p.cmdline()	Ligne de commande → [str]
p.status()	"running", "sleeping", "zombie"...
p.create_time()	Timestamp de création (Unix)
import datetime	
datetime.datetime.fromtimestamp(p.create_time())	
p.username()	Utilisateur propriétaire
p.uids()	UIDs (real, effective, saved) (Unix)
p.gids()	GIDs (real, effective, saved) (Unix)
p.cwd()	Répertoire de travail courant
p.environ()	Variables d'environnement → dict
p.terminal()	Terminal associé (Unix)

CPU :

p.cpu_percent(interval=None)	% CPU (non bloquant, relatif au dernier appel)
p.cpu_percent(interval=1)	% CPU sur 1 seconde
p.cpu_times()	Temps CPU (namedtuple user, system, children_user...)
p.cpu_affinity()	Liste des cœurs autorisés [0, 1, 2, 3]
p.cpu_affinity([0, 1])	Restreint aux cœurs 0 et 1 (setter)
p.cpu_num()	Cœur sur lequel le processus tourne actuellement
p.nice()	Priorité (-20 à 19 sur Unix, 0-6 sur Windows)
p.nice(10)	Modifie la priorité (setter, peut nécessiter root)

MÉMOIRE :

p.memory_info()	Info mémoire (namedtuple)
m = p.memory_info()	
m.rss	Resident Set Size (octets, RAM physique réelle)
m.vms	Virtual Memory Size (espace adressage total)
m.shared	Mémoire partagée (Linux)
m.text	Code exécutable (Linux/macOS)
m.data	Données (Linux/macOS)
m.lib	Bibliothèques partagées (Linux)
m.dirty	Pages dirty (Linux)
m.uss	Unique Set Size (Linux/macOS/Windows) – le plus utile
m.pss	Proportional Set Size (Linux)
m.swap	Swap utilisé (Linux)
p.memory_percent()	% de la RAM totale
p.memory_percent(memtype="rss")	Depuis rss (défaut)
p.memory_full_info()	Infos complètes (uss, pss, swap – plus lent)
p.memory_maps()	Mappings mémoire → liste namedtuple
p.memory_maps(grouped=True)	Groupé par path

FICHIERS ET CONNEXIONS :

p.open_files()	Fichiers ouverts → [namedtuple(path, fd, ...)]
for f in p.open_files():	
f.path	Chemin du fichier
f.fd	File descriptor
f.position	Position courante
f.mode	Mode d'ouverture
f.flags	Flags O_RDONLY etc.
p.connections()	Connexions réseau du processus
p.connections(kind="tcp")	TCP seulement
for conn in p.connections():	
conn.fd; conn.family; conn.type	
conn.laddr; conn.raddr; conn.status	
p.num_fds()	Nombre de file descriptors (Unix)
p.num_handles()	Nombre de handles (Windows)

THREADS ET SIGNAUX :

p.threads()	Threads du processus → [namedtuple]
for t in p.threads():	
t.id	Thread ID
t.user_time	Temps CPU user
t.system_time	Temps CPU system

p.num_threads()	Nombre de threads
p.send_signal(signal.SIGTERM)	Envoie un signal
p.send_signal(signal.SIGKILL)	
p.suspend()	Suspend (SIGSTOP)
p.resume()	Reprend (SIGCONT)
p.terminate()	SIGTERM (gracieux)
p.kill()	SIGKILL (immédiat)
p.wait(timeout=3)	Attend la fin → code retour
p.wait(timeout=None)	Attend indéfiniment

HIÉRARCHIE :

p.parent()	Processus parent → Process
p.parents()	Tous les ancêtres → [Process]
p.children()	Enfants directs → [Process]
p.children(recursive=True)	Tous les descendants
psutil.Process(os.getpid()).children(recursive=True)	

CONTEXT MANAGER :

with psutil.Process(pid) as p:	Libère les ressources automatiquement
print(p.name())	

as_dict() :

p.as_dict()	Tous les attributs → dict
p.as_dict(attrs=["pid", "name", "memory_percent"])	Attributs sélectifs

EXEMPLES :

```
# Top 5 processus par CPU
procs = []
for p in psutil.process_iter(["pid", "name", "cpu_percent"]):
    try:
        p.cpu_percent(interval=None) # initialise
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        pass
    time.sleep(1)
for p in psutil.process_iter(["pid", "name"]):
    try:
        procs.append((p.cpu_percent(), p.info["name"], p.info["pid"]))
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        pass
for cpu, name, pid in sorted(procs, reverse=True)[:5]:
    print(f"{cpu:6.1f}% {name:<20} PID={pid}")
```

[psutil] Informations système globales

DESCRIPTION : Informations globales sur le système (uptime, utilisateurs, boot).
 POURQUOI : Rapports système, monitoring, inventaire.

SYSTÈME :

psutil.boot_time()	Timestamp du dernier démarrage (Unix)
import datetime	
datetime.datetime.fromtimestamp(psutil.boot_time())	
psutil.users()	Utilisateurs connectés → [namedtuple]
for u in psutil.users():	
u.name	Nom de l'utilisateur
u.terminal	Terminal (tty, pts/0...)
u.host	Hôte distant (si SSH)
u.started	Timestamp de connexion
u.pid	PID du processus de login (Linux/macOS)
psutil.os.getpid()	PID du processus courant (= os.getpid())
psutil.version_info	Version de psutil → (5, 9, 0)

CONSTANTES :

psutil.POSIX	True si Unix
psutil.WINDOWS	True si Windows
psutil.LINUX	True si Linux
psutil.MACOS	True si macOS
psutil.OSX	Alias de MacOS
psutil.FREEBSD	True si FreeBSD
psutil.STATUS_RUNNING	"running"
psutil.STATUS_SLEEPING	"sleeping"
psutil.STATUS_DISK_SLEEP	"disk-sleep"
psutil.STATUS_STOPPED	"stopped"
psutil.STATUS_ZOMBIE	"zombie"
psutil.STATUS_DEAD	"dead"

```
psutil.IOPRIO_CLASS_IDLE
psutil.IOPRIO_CLASS_BE
psutil.IOPRIO_CLASS_RT
```

Classe de priorité I/O (Linux)

```
EXAMPLES :
boot = datetime.datetime.fromtimestamp(psutil.boot_time())
uptime = datetime.datetime.now() - boot
print(f"Uptime: {uptime.days}j {uptime.seconds//3600}h")
```

PYNPUT – CLAVIER ET SOURIS

[pynput.keyboard] Contrôle du clavier

DESCRIPTION : Simule des frappes clavier, combinaisons de touches.
POURQUOI : Automatisation de saisie, tests UI, bots, scripts.
CONTEXTE : Remplissage de formulaires, raccourcis, génération de texte.

CONTROLLER :

```
from pynput.keyboard import Controller, Key
kb = Controller()
```

TAPER DU TEXTE :

kb.type("Hello, World!")	Tape une chaîne complète
kb.type("Bonjour\n")	Avec entrée
kb.type("αβγδ")	Caractères Unicode
# type() simule les frappes réelles (plus lent mais réaliste)	

PRESSER / RELÂCHER DES TOUCHES :

kb.press("a")	Presse la touche "a"
kb.release("a")	Relâche la touche "a"
kb.press(Key.space)	Presse Espace
kb.release(Key.space)	
kb.press(Key.enter)	Presse Entrée
kb.press(Key.backspace)	Retour arrière
kb.press(Key.delete)	
kb.press(Key.tab)	
kb.press(Key.esc)	
kb.press(Key.shift)	
kb.release(Key.shift)	
kb.press(Key.ctrl)	Ctrl
kb.press(Key.alt)	
kb.press(Key.cmd)	Touche Windows/Cmd

TOUCHES SPÉCIALES (Key.*) :

Key.space	Espace
Key.enter	Entrée
Key.tab	Tabulation
Key.backspace	Retour arrière
Key.delete	Supprimer
Key.esc	Échap
Key.shift	Shift gauche
Key.shift_r	Shift droit
Key.ctrl	Ctrl gauche
Key.ctrl_r	Ctrl droit
Key.alt	Alt gauche
Key.alt_r	Alt droit
Key.alt_gr	AltGr
Key.cmd	Touche Windows (gauche) / Cmd (macOS)
Key.cmd_r	Touche Windows (droite)
Key.caps_lock	Verr. Maj.
Key.num_lock	Verr. Num.
Key.scroll_lock	Arrêt défilement
Key.f1 ... Key.f20	Touches de fonction F1 à F20
Key.up, Key.down, Key.left, Key.right	Flèches directionnelles
Key.home	Début
Key.end	Fin
Key.page_up	Page précédente
Key.page_down	Page suivante
Key.insert	Insertion
Key.print_screen	Impr. écran
Key.pause	Pause
Key.menu	Menu contextuel

COMBINAISONS DE TOUCHES :

```
# Méthode 1 : press/release manuels
kb.press(Key.ctrl)
kb.press("c")
kb.release("c")
kb.release(Key.ctrl)
# Méthode 2 : context manager pressed() (recommandée)
with kb.pressed(Key.ctrl):
    kb.press("c")
    kb.release("c")
with kb.pressed(Key.ctrl):
    kb.type("a")                                Ctrl+A (tout sélectionner)
with kb.pressed(Key.ctrl):
    kb.type("c")                                Ctrl+C (copier)
with kb.pressed(Key.ctrl):
    kb.type("v")                                Ctrl+V (coller)
with kb.pressed(Key.ctrl):
    kb.type("z")                                Ctrl+Z (annuler)
with kb.pressed(Key.alt, Key.ctrl):
    kb.press(Key.delete)                        Ctrl+Alt+Suppr
with kb.pressed(Key.shift):
    kb.press(Key.home)                         Sélectionne jusqu'au début de la ligne
```

KEYCODE PERSONNALISÉ :

```
from pynput.keyboard import KeyCode
kb.press(KeyCode.from_char("é"))                Caractère spécial
kb.press(KeyCode.from_vk(0x41))                 Virtual key code (A = 0x41)
KeyCode.from_dead("~")                         Touche morte (tilde)
```

EXEMPLES :

```
# Copier-coller
kb = Controller()
with kb.pressed(Key.ctrl):
    kb.press("a"); kb.release("a")              Tout sélectionner
time.sleep(0.1)
with kb.pressed(Key.ctrl):
    kb.press("c"); kb.release("c")              Copier
# Ouvrir un terminal (Linux)
with kb.pressed(Key.ctrl, Key.alt):
    kb.press("t"); kb.release("t")
```

[pynput.keyboard] Écoute du clavier (Listener)
--

DESCRIPTION : Capte les événements clavier en arrière-plan.
POURQUOI : Raccourcis globaux, keylogger éthique, tests, hotkeys.
CONTEXTE : Applications avec raccourcis globaux, monitoring de saisie.

LISTENER BASIQUE :

```
from pynput.keyboard import Listener, Key
def on_press(key):
    try:
        print(f"Touche pressée : {key.char}")    # touche normale
    except AttributeError:
        print(f"Touche spéciale : {key}")        # Key.space, Key.enter...
def on_release(key):
    print(f"Touche relâchée : {key}")
    if key == Key.esc:
        return False                            # False arrête le listener
listener = Listener(on_press=on_press, on_release=on_release)
listener.start()                                Démarre en arrière-plan (thread)
listener.join()                                  Attend que le listener se termine
# Ou en tant que context manager :
with Listener(on_press=on_press, on_release=on_release) as listener:
    listener.join()
```

OPTIONS DU LISTENER :

```
Listener(on_press=f)                            Seulement on_press
Listener(on_release=f)                          Seulement on_release
Listener(suppress=True)                         Bloque les touches du reste du système
Listener(win32_event_filter=f)                  Filtre Win32 (Windows seulement)
listener.start()                                Démarre (thread daemon)
listener.stop()                                  Arrête
listener.join()                                  Attend la fin
```

```
listener.running           True si actif
listener.is_alive()       True si le thread est vivant
```

COMBINAISONS ET ÉTAT DES TOUCHES :

```
from pynput.keyboard import Listener, Key, KeyCode
touches_actives = set()
COMBO = {Key.ctrl_l, KeyCode(char="a")}
def on_press(key):
    touches_actives.add(key)
    if touches_actives >= COMBO:
        print("Ctrl+A détecté !")
def on_release(key):
    touches_actives.discard(key)
# Raccourci avec touches de modification
from pynput import keyboard as kb_mod
def verifier_combo(touches_actives, combinaison):
    return all(t in touches_actives for t in combinaison)
# Hotkey intégré (plus simple)
from pynput.keyboard import HotKey, GlobalHotKeys
with GlobalHotKeys({
    "<ctrl>+<alt>+h": lambda: print("Ctrl+Alt+H pressé !"),
    "<ctrl>+<shift>+s": sauvegarder,
}) as h:
    h.join()
```

GLOBAL HOTKEYS :

```
from pynput.keyboard import GlobalHotKeys
GlobalHotKeys({"<ctrl>+a": action})      Raccourci global
GlobalHotKeys({"<ctrl>+<alt>+t": ouvrir_terminal,
               "<ctrl>+c": copier,
               "<super>+l": verrouiller})
# Syntaxe des touches dans GlobalHotKeys :
# "<ctrl>", "<alt>", "<shift>", "<super>", "<cmd>"
# "<f1>..."<f20>", "<space>", "<enter>", "<esc>"
# "<backspace>", "<delete>", "<tab>", "<up>", "<down>"
# Lettres : "a"..."z", chiffres : "0"..."9"
```

EXEMPLES :

```
# Arrêt propre sur Esc
from pynput.keyboard import Listener, Key
def on_press(key):
    if key == Key.esc:
        return False
with Listener(on_press=on_press) as l:
    l.join()
# Journaliser les frappes (usage éthique : son propre PC)
with open("log.txt", "a") as f:
    def on_press(key):
        try: f.write(key.char)
        except AttributeError: f.write(f"[{key}]")
    with Listener(on_press=on_press) as l:
        l.join()
```

[pynput.mouse] Contrôle de la souris

DESCRIPTION : Contrôle la position, les clics et le défilement de la souris.
POURQUOI : Automatisation d'interfaces graphiques, tests, bots.
CONTEXTE : Tests UI, automatisation, jeux, scripts de bureau.

CONTROLLER :

```
from pynput.mouse import Controller, Button
souris = Controller()
```

POSITION :

```
souris.position           → (x, y) position courante
x, y = souris.position
souris.position = (100, 200)   Déplace vers (100, 200)
souris.move(50, 30)          Déplace de +50, +30 pixels relatifs
souris.move(-10, 0)          Déplace vers la gauche de 10px
```

CLICS :

```
souris.press(Button.left)    Presse bouton gauche
souris.release(Button.left)  Relâche bouton gauche
souris.click(Button.left)    Clic gauche simple (press + release)
```

souris.click(Button.left, count=2)	Double-clic gauche
souris.click(Button.right)	Clic droit
souris.click(Button.middle)	Clic molette (bouton central)
souris.click(Button.x1)	Bouton latéral 1 (si disponible)
souris.click(Button.x2)	Bouton latéral 2 (si disponible)
souris.click(Button.left, count=3)	Triple-clic (sélectionne un paragraphe)

DÉFILEMENT (SCROLL) :

souris.scroll(0, 1)	Défilement vers le bas (dx, dy)
souris.scroll(0, -1)	Défilement vers le haut
souris.scroll(1, 0)	Défilement horizontal droite
souris.scroll(-1, 0)	Défilement horizontal gauche
souris.scroll(0, 3)	Défilement rapide vers le bas

Le signe dépend de la plateforme sous Linux/macOS

BOUTONS (Button.*) :

Button.left	Bouton gauche
Button.right	Bouton droit
Button.middle	Bouton central (molette)
Button.x1	Bouton latéral avant
Button.x2	Bouton latéral arrière
Button.unknown	Bouton inconnu

EXEMPLES :

```
# Clic sur une position précise
souris = Controller()
souris.position = (500, 300)
time.sleep(0.1)
souris.click(Button.left)
# Glisser-déposer (drag and drop)
souris.position = (100, 100)
souris.press(Button.left)
time.sleep(0.1)
for _ in range(20):
    souris.move(5, 5)
    time.sleep(0.01)
souris.release(Button.left)
```

Déplace

Clique

[pynput.mouse] Écoute de la souris (Listener)

DESCRIPTION : Capture les événements souris en arrière-plan.
 POURQUOI : Enregistrement de macros, analytics d'interface, tests.
 CONTEXTE : Outils d'accessibilité, tests automatisés, monitoring UX.

LISTENER :

```
from pynput.mouse import Listener, Button
def on_move(x, y):
    print(f"Souris : ({x}, {y})")
def on_click(x, y, button, pressed):
    action = "pressé" if pressed else "relâché"
    print(f"{button} {action} en ({x}, {y})")
    if not pressed and button == Button.right:
        return False
def on_scroll(x, y, dx, dy):
    print(f"Scroll en ({x},{y}) : dx={dx} dy={dy}")
listener = Listener(on_move=on_move, on_click=on_click, on_scroll=on_scroll)
listener.start()
listener.join()
# Context manager :
with Listener(on_move=on_move, on_click=on_click) as listener:
    listener.join()
```

Arrête au clic droit relâché

OPTIONS DU LISTENER :

Listener(on_move=f)	Seulement les mouvements
Listener(on_click=f)	Seulement les clics
Listener(on_scroll=f)	Seulement le défilement
Listener(suppress=True)	Bloque les événements du reste du système
listener.stop()	Arrête le listener
listener.running	True si actif

SIGNATURES DES CALLBACKS :

on_move(x, y)	x, y : coordonnées (int)
on_click(x, y, button, pressed)	button : Button.*, pressed : bool
on_scroll(x, y, dx, dy)	dx, dy : delta du scroll (int)

```
# Retourner False arrête le listener
```

```
EXEMPLES :
# Enregistrer le clic gauche et sa position
clicks = []
def on_click(x, y, button, pressed):
    if pressed and button == Button.left:
        clicks.append((x, y))
        print(f"Clic #{len(clicks)} : ({x}, {y})")
        if len(clicks) >= 5:
            return False # Arrête après 5 clics
with Listener(on_click=on_click) as l:
    l.join()
print(f"Positions cliquées : {clicks}")
```

Combinaison clavier + souris

DESCRIPTION : Utilisation simultanée des deux listeners et controllers.
POURQUOI : Automatisation complète, enregistrement/replay de macros.
CONTEXTE : Bots, tests UI, outils d'accessibilité.

```
LISTENERS SIMULTANÉS :
from pynput import keyboard, mouse
kb_listener = keyboard.Listener(on_press=on_key_press)
ms_listener = mouse.Listener(on_click=on_mouse_click)
kb_listener.start()
ms_listener.start()
kb_listener.join()
ms_listener.join()

MACRO RECORDER / REPLAYER :
import time
from pynput import mouse, keyboard
from pynput.mouse import Button, Controller as MCtrl
from pynput.keyboard import Key, Controller as KCtrl
# Enregistrement
events = []
t0 = time.time()
def on_click(x, y, button, pressed):
    events.append(("click", time.time()-t0, x, y, button, pressed))
def on_press(key):
    events.append(("key", time.time()-t0, key, True))
m1 = mouse.Listener(on_click=on_click)
k1 = keyboard.Listener(on_press=on_press)
m1.start(); k1.start()
time.sleep(5) # Enregistre 5 secondes
m1.stop(); k1.stop()
# Replay
m = MCtrl(); k = KCtrl()
t0 = time.time()
for evt in events:
    while time.time() - t0 < evt[1]:
        time.sleep(0.001)
    if evt[0] == "click" and evt[5]: # pressed
        m.position = (evt[2], evt[3])
        m.press(evt[4])
    elif evt[0] == "click" and not evt[5]:
        m.release(evt[4])
    elif evt[0] == "key":
        k.press(evt[2])
```

RÉCAPITULATIF COMPLET

— PSUTIL — CPU —	
psutil.cpu_percent(interval, percpu)	% utilisation CPU
psutil.cpu_times(percpu)	Temps CPU (user, system, idle...)
psutil.cpu_times_percent(interval, percpu)	Temps en %
psutil.cpu_stats()	ctx_switches, interrupts, syscalls
psutil.cpu_freq(percpu)	Fréquence (current, min, max) MHz
psutil.cpu_count(logical)	Nombre de cœurs
psutil.getloadavg()	Charge (1min, 5min, 15min)

— PSUTIL – MÉMOIRE —

psutil.virtual_memory()	total, available, used, free, percent...
psutil.swap_memory()	total, used, free, percent, sin, sout

— PSUTIL – DISQUES —

psutil.disk_partitions(all)	device, mountpoint, fstype, opts
psutil.disk_usage(path)	total, used, free, percent
psutil.disk_io_counters(perdisk, nowrap)	read/write count, bytes, time

— PSUTIL – RÉSEAU —

psutil.net_io_counters(pernic, nowrap)	bytes_sent/recv, packets, err, drop
psutil.net_if_addrs()	Adresses par interface
psutil.net_if_stats()	isup, duplex, speed, mtu
psutil.net_connections(kind)	fd, family, laddr, raddr, status, pid

— PSUTIL – PROCESSUS —

psutil.pids()	Liste de tous les PIDs
psutil.process_iter(attrs)	Itère sur tous les processus
psutil.pid_exists(pid)	Test d'existence
psutil.Process(pid)	Objet processus
p.pid, p.ppid(), p.name(), p.exe(), p.cmdline()	
p.status(), p.create_time(), p.username()	
p.cpu_percent(interval), p.cpu_times(), p.cpu_affinity(), p.nice()	
p.memory_info(), p.memory_percent(), p.memory_full_info()	
p.open_files(), p.connections(kind), p.num_fds()	
p.threads(), p.num_threads()	
p.parent(), p.parents(), p.children(recursive)	
p.send_signal(), p.suspend(), p.resume(), p.terminate(), p.kill(), p.wait()	
p.as_dict(attrs)	

— PSUTIL – SYSTÈME —

psutil.boot_time()	Timestamp du démarrage
psutil.users()	Utilisateurs connectés
psutil.LINUX, psutil.WINDOWS, psutil.MACOS	Détection de plateforme

— PYNPUT – KEYBOARD CONTROLLER —

kb = keyboard.Controller()	
kb.type("texte")	Tape du texte
kb.press(Key.enter) / kb.release(Key.enter)	Presse/Relâche
kb.press("a") / kb.release("a")	Touche normale
with kb.pressed(Key.ctrl): kb.type("c")	Combinaison

— PYNPUT – KEYBOARD LISTENER —

```

Listener(on_press=f, on_release=f, suppress=bool)
GlobalHotKeys({"<ctrl>+a": f, "<f1>": g})
HotKey(touches, action)
→ on_press(key): key.char ou Key.*
→ on_release(key): retourner False pour arrêter

```

— PYNPUT – MOUSE CONTROLLER —

m = mouse.Controller()	
m.position	getter/setter (x, y)
m.move(dx, dy)	
m.click(Button.left, count=1)	
m.press(Button.left) / m.release(Button.left)	
m.scroll(dx, dy)	

— PYNPUT – MOUSE LISTENER —

```

Listener(on_move=f, on_click=f, on_scroll=f, suppress=bool)
→ on_move(x, y)
→ on_click(x, y, button, pressed)
→ on_scroll(x, y, dx, dy)
→ retourner False dans un callback arrête le listener

```

— PIÈGES COURANTS —

```

cpu_percent() 1er appel → 0.0 (pas de référence précédente)
process_iter() sans attrs → très lent (charge tout)
Process(pid) après pid_exists() → race condition possible
pynput sur macOS → nécessite autorisation Accessibilité
pynput Linux sans X11 → DISPLAY non défini → erreur
Listener.suppress=True → bloque les events pour toutes les applis
on_press retourne False → ARRÊTE le listener (ne pas oublier)
souris.scroll(0, 1) → direction peut varier selon la plateforme
kb.type() → plus lent que press/release (simule des vraies frappes)
memory_info().rss → RSS peut inclure des pages partagées (surestimation)

```

memory_full_info().uss → plus précis mais plus lent à obtenir
disk_io_counters() → compteurs cumulatifs depuis le démarrage (faire des deltas)
net_io_counters() → idem, toujours cumulatifs

=====